Alex Castillo González

Applied AI Engineer · Python / LLM

Santo Domingo, Dominican Republic — UTC−4 · Independent contractor (Spanish passport) · Spanish (native), English (professional) alex.castillog33@gmail.com · github.com/pcbeingused333 · linkedin.com/in/alexcastillogonzalez · portfolio-alexgonzalez33.vercel.app

SUMMARY

Applied AI engineer in Python: two retrieval and agent systems in production, each shipping the harness that measures it, and **fifty merged pull requests** this year into the frameworks they run on. My main project answers over the text of the **GDPR** under a constraint generic RAG ignores — every statement names the provision it came from, and the system declines when the source does not cover the question. In both projects the harness found defects the tests did not. Fullstack across Python, TypeScript and Ruby; twelve of those merged fixes are in **Haystack** and its integrations (deepset's RAG and agent framework), concurrency and async/sync-parity defects, and nine in **LangChain.rb**.

SKILLS

AI / LLM — Python · LLM APIs (Groq, OpenAI-compatible) · LangChain · LangGraph · ReAct agents and tool use · **Model Context Protocol (MCP)**: building servers and clients · **LLM evaluation**: known-answer datasets, retrieval metrics (hit@1, recall@k, MRR), **abstention and unsupported-claim rate**, agent trajectory scoring, LLM-as-judge for faithfulness / relevancy / correctness · RAG: chunking strategy, retrieval tuning, **structure-aware citation at the provision level** · cross-lingual retrieval

Regulatory / legal text — parsing legislative sources into citable provisions (EUR-Lex / Official Journal markup) · citation integrity as a design constraint · evaluating refusal on out-of-corpus questions · GDPR structure (data subject rights, controller and processor obligations, breach notification, administrative fines)

Data & retrieval — embeddings (`sentence-transformers`, BGE) · FAISS · PostgreSQL + `pgvector` · PDF ingestion pipelines · structured corpus construction

Backend & web — Python · pytest · Ruby on Rails · PostgreSQL · REST APIs · JSON/API integrations (Jira API) · TypeScript · Next.js · React · Tailwind

Infra & tooling — **AWS** (Lambda container images, DynamoDB, ECR, IAM least-privilege, CloudWatch) · **Terraform** · **CI/CD with GitHub Actions, OIDC federation** · Docker · Vercel · Git · AI-assisted development (Claude Code, Cursor)

EXPERIENCE

Churrería Calderón — Founder & owner-operator · Toronto, Canada

Sep 2025 – Jul 2026 *(business closed July 2026)*

- Moved my business from Spain to Toronto on my own: leased premises in Kensington Market, imported the machinery, registered the business and obtained every permit to operate legally. Opened in December 2025 and ran it with no hires — production, service, purchasing, cash and compliance — until it closed in July 2026.

Family churrería business — Manager · Catalonia, Spain

2023 – Aug 2025 · mainly summer seasons *(and in the same business before that)*

- Managed the family business, mainly in the summer seasons — production, service, purchasing and cash — and took it from a mobile trailer to fixed premises, rebuilding the operation around a different site, hours and customer base.

Le Wagon — Part-time Programming Teacher · Remote (France)

Oct 2022

- Taught the new part-time flex cohort of the fullstack bootcamp — Ruby, OOP, SQL and PostgreSQL, HTML/CSS/JavaScript, Rails — through exercises and live debugging.

TECNOBIT (Grupo Oesía) — Fullstack Developer · Valdepeñas, Spain

Aug 2022 – Nov 2022 · On-site

- Shipped features into a long-running internal application maintained by a team of five, across a codebase distributed over both GitHub and GitLab.
- **Backend:** Ruby routines pulling data from JSON files and the **Jira API**, persisting it to PostgreSQL with multi-stage validation gating document generation and downloads. **Frontend:** form-driven pages and PostgreSQL-backed views with role-dependent data, surfacing subprocess state so users could follow a download to completion.

SELECTED PROJECTS

Business Ops Agent — MCP server + agent, with trajectory evaluation

Live demo · Code

A **Model Context Protocol server** exposing a business's operations (catalog, booking capacity, stock, quoting, orders) as tools any MCP client can call — Claude Desktop, Cursor, or the LangGraph agent bundled with it. The agent carries no business rules: tools are discovered at runtime, so adding one requires no agent change.

- Built an evaluation harness that scores **tool trajectories**, not just answers: which tools were called, in what order, with which arguments, and whether every figure in the reply traces back to a tool result — deterministic, no judge model, so it holds even when a fabricated number happens to be correct. It caught the agent answering with zero tool calls, and caught it **intermittently**, which single-run testing misses. 11/12, mean 0.98.
- **Deployed to AWS** as a remote MCP server — Lambda container behind a Function URL, DynamoDB single-table store, least-privilege IAM, all in Terraform. Storage sits behind an interface, so the same server runs on SQLite locally and DynamoDB in production.
- Python, MCP 2.0, LangGraph, Groq, AWS (Lambda, DynamoDB, ECR, IAM), Terraform, Docker, pytest (161 tests). CI/CD on every push via GitHub Actions authenticating with **OIDC** rather than a stored key, scoped to one branch and unable to apply infrastructure.

Ask the GDPR — retrieval over regulation, with citations that can be checked

Live demo · Code

LangGraph agent over the full text of the GDPR. Every answer names the provision behind it — `Art. 33(1)`, not a page number — and the system declines when the source does not cover the question. Two modes behind one flag: an in-memory FAISS demo on a free 1 GB container, and a pgvector-backed production path.

- **Made the citation structural rather than incidental.** A regulation is cited by article and paragraph; the page a provision lands on is an artefact of typesetting. So the corpus is not a PDF: a builder parses the Official Journal text from EUR-Lex into **414 provisions** carrying article, paragraph and chapter as metadata, and the chunk size was chosen so **97% survive as exactly one chunk** — a chunk straddling Art. 33(1) and 33(2) cites the wrong paragraph.
- **Built an eval for the answers that should never be given.** In legal text a retrieval miss announces itself; an invention is fluent and indistinguishable from a correct answer. The harness scores refusal against questions the Regulation does not answer but every model has read about — adequacy decisions, Schrems II, a CCPA penalty — plus a deterministic check for citations that appear in the answer and never in the passages.
- **Re-ran every measurement when the corpus changed, and one result reversed.** Embedding the article heading hurt retrieval under the old embedding model and helped under the new one; carrying the first conclusion forward would have shipped the worse setting on the strength of real evidence. The model swap was worth 8/20 → 13/20 at rank 1 for 42 MB, against a hard 1 GB ceiling.
- Python, LangChain, LangGraph, Groq, FAISS, pgvector, Streamlit, Docker, GitHub Actions, pytest (72 tests). CI runs headlessly on every push, including a boot test of the app — the host redeploys straight from `main`, so the suite is the only gate before the demo.

ragcite — the evaluation layer of the two projects above, as a library

Code

Retrieval metrics, citation grounding, abstention scoring and tool-trajectory scoring, extracted from the two harnesses above and generalized. No LangChain, vector store or LLM client is imported: you bring the retriever and the judge, `ragcite` scores what they return.

- **It reproduces the numbers of the project it came from, and asserts it.** The dogfood example scores one FAISS index twice — once with `ragcite`, once with the original project's own independent metrics code — and exits non-zero if they disagree. They match exactly (hit@1 13/25, recall@k 17/25, MRR 0.59).
- Zero runtime dependencies, MIT, 155 tests, CI on Python 3.9–3.12. `--min-hit-at-1` turns a run into a CI gate that exits 2 on a regression.

OPEN SOURCE

Merged — fifty pull requests this year. Twelve across `deepset-ai/haystack` and its integrations, each with a regression test I verified fails with the fix reverted — concurrency defects on the async path (four of them invisible to the test suite for the same reason, written up together: Four concurrency bugs on Haystack's async path), async/sync-parity gaps, and components dropping an `__init__` parameter from `to_dict` so the setting silently reverted on reload, found by an AST audit of `__init__` against `to_dict` (#3808). Also merged: nine in `LangChain.rb`, edge cases that crashed a caller instead of degrading; a documentation fix in `pydantic-ai` for an eval case that could not fail; Claude-compatibility fixes in `Tencent/WeKnora` (two), `hostedgpt` and `chatlas`; two in `pyfenn/fenn`; twenty across Ruby tooling, a mailer gem, a static site generator and a Rails app; a ranking-validity fix in `opik`; and a role-alias fix in `Arize-ai/phoenix`.

Open — five in `llama-index-core` and four in `deepeval` — metric/scoring correctness bugs — plus open fixes in `ragas`, `giskard` and `guardrails`, and across Ruby tooling. Defects I only reported, each with a standalone reproduction, are triaged and taken up the same way: `pydantic-ai` #7927 — `LLMJudge` grading a `bytes` output rendered as one decimal byte per line, no error — is accepted for implementation, and the cached lock above was triaged `P3` and closed by my own PR.

EDUCATION

Le Wagon — Fullstack Web Development bootcamp, 2022. Self-directed study since.